Computer Networks 289 (2026) 112724

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

Towards adaptive and robust website fingerprinting defense with multi-agent deep reinforcement learning

Yufei Zhu [a], Le Tian [a,b,c], Yuxiang Hu [a,b,c,*], Xinglong Pei [a], Qi Zhan [a], Qilong Huang [a]

[a] Information Engineering University, Zhengzhou, China
[b] Key Laboratory of Cyberspace Endogenous Safety&Security of Henan Province, Zhengzhou, China
[c] Key Laboratory of Cyberspace Security Ministry of Education of China, Zhengzhou, China

ARTICLE INFO

Keywords:
Anonymous communication
Website fingerprinting defense
Privacy protection
Cyberspace security
Multi-agent deep reinforcement learning

ABSTRACT

With the increasing threat of website fingerprinting (WF) attacks against anonymous communication systems, a variety of defense strategies, such as randomization, regularization, multipath, and adversarial defenses, have demonstrated promising results under specific conditions. However, existing defense strategies still face three critical limitations: (1) limited adaptability to evolving attacks and dynamic environments; (2) high computational costs due to complex mechanisms; (3) inconsistent performance across different attacker models and traffic scenarios. To enhance the adaptive capacity and adversarial robustness of defense strategies, we propose RAMAD (Robust adaptive multi-agent defense, RAMAD), an adaptive traffic splitting defense strategy based on multi-agent deep reinforcement learning (MADRL), which models the defender and attacker as two agents engaged in a dynamic adversarial game, enabling evolution and strategic optimization of both agents. Joint training is implemented via the multi-agent proximal policy optimization (MAPPO) algorithm. While preserving communication performance, RAMAD effectively disrupts the attacker's fingerprinting behavior by dynamically adjusting the traffic allocation ratio across multiple paths. System simulation evaluations are conducted based on real Tor traffic datasets and various typical attack models. The results show that RAMAD can reduce the attack success rate to 13.13%, representing an approximately 80% decrease, while balancing low overhead and the high path allocation entropy, showing strong adaptability and defense performance. Compared with traditional defense methods such as Tamaraw, WTF-PAD, and FRONT, this strategy achieves a good balance between security and performance, demonstrating its potential and scalability as an intelligent anonymous communication defense mechanism.

1. Introduction

With the rapid advancement of internet technologies, user privacy protection and data security have become central concerns in the digital society. Anonymous communication systems, such as the Tor network [1,2], have emerged as a key implementation of privacy-enhancing technologies, offering a solution to safeguard users' online security [3]. However, it is noteworthy that while these systems effectively encrypt communication content, they still generate substantial side-channel metadata during transmission—such as packet size, timing, and request frequency [4]. This metadata remains exploitable by attackers, potentially revealing user behavioral patterns and posing new threats to privacy protection. Website fingerprinting attacks exploit such metadata by using machine learning models to reconstruct information about visited websites, posing a severe threat to the security of anonymous communication systems [5]. Research indicates that deep learning-based website fingerprinting attacks achieve over 95% accuracy on public datasets, rendering traditional static defense strategies increasingly ineffective [6].

To address this challenge, researchers have proposed four mainstream categories of defense strategies: randomization, regularization, adversarial, and multipath defenses [7]. Randomization defenses aim to obfuscate traffic features by injecting noise, but this approach often incurs significant bandwidth overhead [8,9]. Regularization defenses attempt to uniform traffic patterns, yet suffer from high latency and poor resistance against deep learning-based attacks [10,11]. Adversarial defenses generate perturbed adversarial samples to mislead attack classification models [12], but their effectiveness depends heavily on prior knowledge of the attack models and typically lacks real-time adaptability [13,14]. Multipath defenses disrupt attackers' monitoring of global traffic information by distributing traffic across multiple

* Corresponding author at: Information Engineering University, Zhengzhou, China.
E-mail address: huyuxiangchn@163.com (Y. Hu).

https://doi.org/10.1016/j.comnet.2026.112724
Received 26 October 2025; Received in revised form 16 August 2026; Accepted 27 August 2026
Available online 9 September 2026
1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

circuits, offering low overhead and high adaptability, and making it a highly promising research direction [15].

Despite these advances, existing methods still exhibit several limitations, including poor adaptability, high computational cost, and unstable defense performance. Firstly, many current defenses rely on predefined rules, which adversaries can gradually learn and adapt to, thereby bypassing protection mechanisms. This makes them ineffective in complex and dynamic attack scenarios. Secondly, some defense mechanisms require substantial computational resources, such as dynamic traffic padding or frequent packet reshuffling, which impose considerable pressure on both the computation and the network infrastructure during deployment. Finally, defense strategies are typically designed statically, lacking flexibility and potentially exhibiting unstable performance against complex attacks. These issues fundamentally stem from traditional methods' lack of environmental awareness and strategy adaptability, making them insufficiently robust against evolving threats.

To address the aforementioned issues, we propose RAMAD, an adaptive traffic splitting defense strategy based on multi-agent deep reinforcement learning (MADRL) [16]. RAMAD centers on a "defender–attacker" game theory framework, modeling both parties as dynamically adversarial reinforcement learning agents. It employs the multi-agent proximal policy optimization (MAPPO) algorithm for training to optimize agent strategies. Within this framework, the defender agent adaptively adjusts the traffic splitting policies in response to the changes in the network conditions and attacker behaviors, thereby obscuring website access patterns and enhancing defense effectiveness. In contrast, the attacker agent attempts to extract traffic fingerprints by analyzing partially monitored traffic data to identify the websites users visit. During training, the defender and the attacker agents engage in continuous interaction and learning. Upon convergence, the defender agent identifies optimal defense strategies for diverse attack scenarios. In summary, the central idea of this work is modeling the attack–defense interaction through Markov games. By leveraging multi-agent deep reinforcement learning algorithms to continuously refine strategies, it constructs an intelligent defense system that integrates perception, decision-making, and adversarial adaptation.

The main contributions of our work are summarized as follows:

(1) We are the first to formalize the WF attack and defense interaction as a distributed partially observable Markov decision process (Dec-POMDP), clearly describing the adversarial mechanism between the defender and the attacker, thus providing a theoretical foundation for subsequent reinforcement learning-based policy optimization.
(2) We propose RAMAD, a novel MADRL framework for WF defense, modeling the defender and attacker as two learnable heterogeneous agents. By incorporating centralized training with decentralized execution (CTDE) and adversarial training, the defender's policy evolves adaptively in response to the attacker's strategy.
(3) To balance defense strength and communication efficiency, we design a multi-objective reward function that jointly considers attack detection accuracy, traffic balance, and path cost. This enables the defender to learn strategies that effectively trade off between privacy protection and network performance.
(4) Comprehensive experiments on both simulation environments and real-world Tor traffic datasets demonstrate that RAMAD significantly reduces attack accuracy — by over 80% compared to no defense — and outperforms existing mainstream approaches in maintaining low communication overhead.

The rest of this paper is organized as follows. Section 2 reviews previous works on WF attacks and defenses, as well as the applications of Deep Reinforcement Learning (DRL) in the Tor network. Section 3 presents preliminary knowledge, including the WF threat model and MADRL fundamentals. Section 4 details the overall architecture of our proposed defense framework, RAMAD, along with its core algorithm and training procedure. Section 5 provides comprehensive experimental evaluations, and Section 6 concludes the paper.

2. Related works

In this section, we introduce and summarize prior work on WF attacks and defenses, as well as the applications of DRL in the Tor network.

2.1. Website fingerprinting attacks

WF attack is a traffic analysis attack targeting anonymous communication systems [17]. Attackers infer information about websites accessed by users by analyzing metadata characteristics leaked in encrypted communications, such as packet size, timing, and direction. Based on the technical methods employed, WF attacks can be categorized into two types [18]: (1) traditional machine learning (ML)-based WF attacks, and (2) deep learning (DL)-based WF attacks.

(1) Traditional machine learning-based WF attacks. Early WF research primarily relied on manual feature engineering and traditional classifiers to identify website traffic patterns.

Wang et al. [19] proposed a method based on the k-Nearest Neighbor (k-NN) classifiers. It optimized classification performance by dynamically adjusting the weights of traffic features (e.g., packet length, sequence, burst patterns), and enhanced discrimination capabilities by incorporating multimodal feature sets (e.g., total transmission size, initial packet sequence, packet direction concentration). Through iterative feature weight updates, irrelevant features were reduced, improving classification accuracy. This approach achieved a true positive rate (TPR) of 85% in open-world scenarios with a false positive rate (FPR) of only 0.6%.

Panchenko et al. [20] proposed a feature extraction method based on cumulative traffic representation, combined with Support Vector Machines (SVM) for classification. They transformed traffic sequences into cumulative packet size curves and sampled a fixed number of feature points to capture dynamic traffic variations. On the Alexa Top 100 dataset, accuracy reached 92%, with TPR at 96.6% and FPR at 9.6% in open-world scenarios. Hayes et al. [21] proposed the k-fp attack, which utilized a random forest (RF) classifier. This method converted traffic features into fixed-length fingerprint vectors and employed Hamming distance for classification. It demonstrated strong resilience under multiple defense mechanisms such as BuFLO, Tamaraw, and Decoy Pages, achieving attack accuracies ranging from 21% to 90%, significantly outperforming prior methods such as k-NN and CUMUL (which ranged from 8% to 75%).

(2) Deep learning-based WF attacks. Although traditional machine learning-based methods possess effectiveness in certain scenarios, their limited feature extraction capabilities result in poor performance when confronted with complex traffic patterns and advanced defense strategies. In recent years, deep learning models have demonstrated stronger generalization and adaptability in WF attacks due to their end-to-end feature extraction capabilities, significantly improving attack accuracy [22].

Sirinam et al. [6] proposed the Deep Fingerprinting (DF) attack, which leverages convolutional neural networks (CNNs) to automatically extract traffic features. This method transforms the raw traffic sequence into a two-dimensional matrix and utilizes multi-scale convolution kernels to extract both local and global features. On a dataset comprising 100 monitored websites and 5000 unmonitored websites, DF achieved a True Positive Rate (TPR) of 98.5% with a False Positive Rate (FPR) of only 0.06%. Even against the WTF-PAD defense, TPR remained at 91%, far outperforming k-NN (only 35%). Shen et al. [23] proposed the RF attack, based on Traffic Aggregate Modeling (TAM), which jointly models temporal and directional traffic characteristics.

The approach strengthens both the adversarial robustness and feature representation capability of the attack, achieving enhanced accuracy under various network conditions.

Rahman et al. [24] introduced the Tik-Tok attack, which systematically incorporates temporal information for the first time by combining packet direction and timestamp features. By combining packet direction and timestamp features, it captures the burst-like temporal structure of traffic to enhance attack accuracy. Experiments demonstrate that Tik-Tok outperforms traditional direction-only methods across diverse attack scenarios. Similarly, Gong et al. [25] proposed WFCAT to address the insufficient utilization of temporal features in existing. They innovatively designed intermediate-grained temporal features based on inter-arrival time (IAT) histograms and developed a CNN model tailored to such features. By employing multi-scale convolutional kernels to capture temporal patterns at different resolutions and incorporating channel attention to adaptively weight feature importance, their approach significantly improves effective feature extraction in noisy traffic environments.

2.2. Website fingerprinting defenses

To mitigate the growing threats of de-anonymization posed by increasingly sophisticated WF attacks, researchers have proposed a wide range of defense strategies [26]. These strategies can be broadly categorized into four main types: Randomization, Regularization, Adversarial, and Multipath defenses [7]. This section provides a systematic review of representative methods within each category, highlighting their key mechanisms, advantages, and primary limitations. Table 1 presents a comparison between RAMAD and these representative methods in terms of efficiency, resource consumption, and adaptability.

(1) Randomization-based defenses. These methods introduce randomness into the network traffic flow (e.g., by forging packets or adding delay perturbations) to disrupt identifiable traffic patterns, thereby reducing the effectiveness of attack models. For example, WTF-PAD [27] and FRONT [28] conceal the original features by perturbing inter-packet intervals and altering flow directions, respectively. The advantage of such approaches lies in their relatively simple implementation and minimal requirement for modifying traffic content. However, these defenses often fail against deep learning attacks, and the perturbation intensity is highly coupled with communication overhead, leading to bandwidth waste.

(2) Regularization-based defenses. These methods increase the difficulty for attack models to distinguish traffic by integrating and standardizing features across different websites. A representative method, Tamaraw [29], normalizes traffic through fixed packet intervals and specific directional sequences. While theoretically enhancing robustness, these approaches incur high communication costs in practice. Specifically, fixed-rhythm transmission strategies introduce significant latency and additional bandwidth demands. Moreover, uniform feature patterns are easily captured by deep models, leading to diminished long-term defense effectiveness.

(3) Adversarial defenses. Adversarial defenses design deceptive perturbations in the traffic flow to mislead attack models. Representative methods such as Mockingbird [30] and WF-GAN [31] generate traffic perturbation samples based on target optimization and generative adversarial networks (GANs), respectively, achieving strong defense at low overhead. However, these approaches generally rely on offline training processes and exhibit strong prior dependencies on attack model structures. As a result, they lack real-time adaptability and involve high computational costs during training, making it difficult to meet the low-latency and high-efficiency requirements of practical anonymous communication systems. Huang et al. proposed WF-A2D [32], an asymmetric adversarial defense for HTTP/3. It employs a two-stage cascading adversarial learning strategy: first generating insertion position vectors via the IPG algorithm based on packet direction patterns, then generating perturbation range vectors via the PPRG algorithm based on packet length patterns, achieving packet-level real-time traffic obfuscation. Leveraging QUIC's padding frames and reliable message delivery, WF-A2D performs packet insertion, padding, and splitting independently on the client side without requiring proxy nodes or server-side restoration, enabling proxy-free end-to-end privacy protection.

Table 1
Characteristics of WF defense strategies.

Defense strategy	Efficiency	Resource consumption	Adaptability
WTF-PAD [27]	✗	✓	✓
FRONT [28]	✗	✓	✗
Tamaraw [29]	✓	✗	✗
WF-GAN [31]	✓	✗	✗
Minipatch [34]	✓	✓	✗
RAMAD (Proposed)	✓	✓	✓

Efficiency: Whether the defense can defeat attacks effectively.
Resource Consumption: The computational and bandwidth cost required by the defense strategy.
Adaptability: Whether the defense adapts to real network traffic and different attack models.

(4) Multipath defenses. Multipath methods disrupt attackers' ability to model global traffic patterns by distributing traffic across multiple transmission paths. For example, HyWF [33] probabilistically allocates traffic across two internet connections provided by different Internet service providers (ISPs), reducing attacker's ability to obtain complete traffic features through a single connection. Compared to other defense mechanisms, multi-path defense requires no additional pseudo-packets or latency introduction. It demonstrates significant advantages in bandwidth and latency control, offering low deployment costs and excellent communication performance, thereby exhibiting high potential in practical applications. However, its effectiveness is constrained by specific deployment environments and suffers from limitations such as strong protocol dependency and complex configuration processes.

2.3. Applications of DRL in the Tor network

In recent years, the application of Deep Reinforcement Learning (DRL) in anonymous communication systems has gradually gained attention [35,36], owing to its adaptability and potential for optimizing complex, dynamic environments such as the Tor network. By enabling agents to adjust decision-making strategies in response to real-time environmental feedback, DRL has shown promise in addressing two key challenges in Tor: bandwidth scheduling and WF defense.

(1) Bandwidth scheduling and resource optimization. As one of the most widely used anonymous communication networks in the world, Tor suffers from the persistent issue of unfair circuit bandwidth allocation, which can lead to congestion and performance degradation. To address this, Lamiaa Basyoni et al. [37] proposed the QDRL framework, which formulates Tor's circuit scheduling as a resource allocation problem under Quality of Service (QoS) constraints. They employed the Deep Deterministic Policy Gradient (DDPG) algorithm to design a reinforcement learning agent. The state space includes circuit queue status and kernel buffer states, while the action space consists of connection write limits and circuit-level resource allocations. A custom reward function, which balances throughput improvement and penalty for constraint violations, is used to guide training. Experimental results demonstrate that QDRL significantly improves overall throughput, reduces latency, and enhances fairness compared with traditional heuristic-based schedulers. These results highlight the potential of DRL-based methods to dynamically optimize resource allocation and improve the performance of large-scale anonymous communication networks.

(2) Dynamic website fingerprinting defenses. The rapid advancement of deep learning-based WF attacks poses increasing challenges to conventional, static defense strategies in Tor. To overcome the

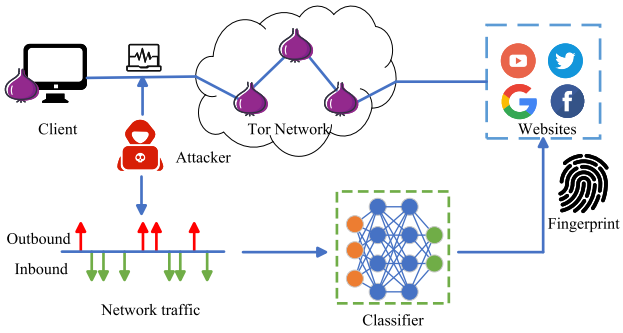


Fig. 1. Website fingerprinting attack threat model.

limitations of manual and inflexible defenses, researchers have begun to explore reinforcement learning-based dynamic defense mechanisms, which adaptively adjust their behavior in real time according to network conditions and attack feedback. Meiyi Jiang et al. [38] proposed KimeraPAD, a low-overhead, real-time defense mechanism based on deep reinforcement learning. The system trains an agent to dynamically insert dummy packets during traffic transmission, thereby confusing the attacker's classifier and disrupting recognizable traffic patterns. In KimeraPAD, the state space captures the current traffic sequence characteristics, and the action space defines the possible positions and directions for dummy packet insertion. The reward function is designed to minimize bandwidth overhead while maximizing disruption to the attacker's classification accuracy. Through this adaptive learning process, KimeraPAD effectively enhances the flexibility and resilience of WF defenses, demonstrating the feasibility of applying DRL to counteract intelligent, data-driven attacks in anonymous communication systems.

Most existing methods are based on single-agent DRL, which can achieve strategy optimization to a certain extent. However, these methods exhibit significant limitations in the context of WF defense, as they assume that the behaviors of other participants remain fixed, making it difficult to effectively counter the dynamic changes and game-theoretic characteristics of attackers' strategies. Given the high uncertainty and diversity of attack behaviors, defenders must possess rapid response capabilities. Therefore, introducing MADRL emerges as a more promising research direction.

3. WF attack–defense framework

3.1. WF threat model

As shown in Fig. 1,We consider a passive website fingerprinting attacker, located between the client and the entry node. The potential attacker could be a listener in the local user network (e.g., a malicious Wi-Fi operator), an Internet Service Provider (ISP), or an intermediate network entity in the routing path (e.g., Autonomous Systems (AS), exchange nodes). Unlike active attackers, this type of attacker does not actively tamper with, inject, or drop Tor traffic, nor does it decrypt any packet contents [39]. Instead, it performs website fingerprinting attacks by passively observing the timing features, packet size, packet direction, and other metadata of the encrypted traffic [2]. We assume that the attacker can observe traffic between the client and a limited number of entry nodes, but cannot monitor the entire Tor network. This assumption reflects a more realistic threat scenario than global passive adversaries.

Website fingerprinting attacks are typically modeled as a supervised learning classification task, where the attacker extracts traffic features and trains a classifier to identify the target website based on traffic patterns, and usually follow three main steps: **(1) Data collection and labeling.** The attacker collects a large number of Tor traffic generated by visiting known websites. Each traffic sample is labeled with the corresponding website identity to build a training dataset. **(2) Feature extraction.** From these raw traffic traces, the attacker extracts key fingerprinting features, including: packet direction (inbound/outbound), packet size, and the intervals between adjacent packets. **(3) Model training and inference.** Traditional machine learning or deep learning methods are employed to train the classifier, enabling it to learn the traffic feature patterns of different websites. Once trained, the attacker can classify and infer the monitored traffic in real-world scenarios to identify the websites being accessed by users.

Website fingerprinting attacks are commonly evaluated in the following two scenarios: **(1) Closed-world scenario.** The attacker predefines a fixed set of websites, assuming that the user only visits these websites, and collects all traffic fingerprints for training. This setup allows for higher identification accuracy and is suitable for evaluating the upper limits of fingerprint classifier performance. **(2) Open-world scenario.** The user can visit any website, and the attacker only has access to the fingerprints of a few websites of interest. Compared to the closed-world scenario, the open-world scenario introduces more uncertainty, significantly increasing the difficulty for the attacker to identify the target website. To avoid overestimating the effectiveness of defense strategies, this study adopts the closed-world setup for adversarial training and evaluation, ensuring that the defense mechanisms remain robust even against the strongest attacker models.

Beyond the standard closed-world website fingerprinting attacker, we further consider a more practical adaptive adversary. In real-world deployment, once the attacker observes that the deployed defense has altered the statistical properties of the traffic, it may retrain or fine-tune its classifier on the defended traffic and adjust its attack strategy accordingly. Moreover, the attacker may combine multiple base models via majority voting to enhance attack robustness. This more realistic threat model is systematically evaluated in our robustness analysis.

3.2. Problem formulation

The core issue addressed in this study is how to design an adaptive traffic splitting strategy that effectively mitigates against the de-anonymization threat posed by WF attacks, while maintaining acceptable communication performance. Traditional WF defenses are largely based on static, rule-based mechanisms, which struggle to adapt to the dynamic evolution of attacker strategies. As a result, their defense effectiveness is often unstable or limited in real-world environments.

To enhance defense adaptability, we propose an adaptive traffic splitting strategy that probabilistically distributes user access requests across multiple available circuits. By randomizing circuit selection, the defender disrupts the attacker's ability to observe and extract traffic features, thereby degrading classification accuracy. Meanwhile, such a strategy must carefully balance the trade-off between disrupting the attacker's classification accuracy and maintaining network communication performance. Therefore, it can be formalized as the following optimization problem: minimize the attacker's identification accuracy subject to communication performance constraints.

Let the user u access the target website v, and the set of available circuits provided by the Tor network are denoted as c. Define the strategy variable $p = [p_1, p_2, \ldots, p_n]$, where p_i represents the probability of allocating a portion of the traffic to circuit i. The attacker's classification model can be abstracted as a classifier, where the input is the passively observed traffic feature vector (e.g., timing, packet direction), and the output is the predicted website label. Let $\hat{y}(p)$ represent the attacker's classification accuracy under the traffic splitting strategy p. Then, the defender's optimization objective is : $\min_p \hat{y}(p)$, subject to the following constraints:

(1) Traffic integrity constraint: $\sum_i p_i = 1$, which ensures that the traffic is fully allocated.

(2) Circuit bandwidth constraint: $\sum_u b_u \leq B_i, \quad \forall i \in c$, where b_u represents the traffic demand of user u, and B_i is the bandwidth capacity of circuit i.

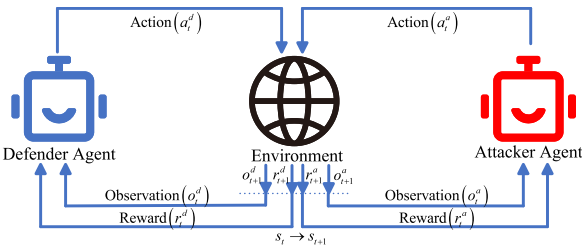


Fig. 2. MADRL interaction model.

(3) Communication delay constraint: $\sum_i d_i \leq D_{max}$, where d_i represents the delay for circuit i, and D_{max} is the maximum acceptable delay for the user.

This optimization problem is difficult to model explicitly, especially when the attacker is capable of strategy learning and dynamically adjusting the classifier parameters. The defender faces a highly non-convex, unstable dynamic optimization problem. Furthermore, considering that the attacker has adaptive learning capabilities, the classification strategy may constantly update with changes in the defense mechanism. In this setup, the problem essentially evolves into a two-agent adversarial game, where the defender and the attacker agents engage in strategy iteration within the same environment, each attempting to minimize and maximize classification accuracy, respectively: $\min_{\pi_d} \max_{\pi_a} \hat{y}$, where π_d represents the defender's strategy (i.e., how to generate the splitting vector p), and π_a represents the attacker's strategy (i.e., how to select the attack model and monitoring paths).

During this adversarial game, both agents iteratively update their strategies to optimize their respective objectives. The defender aims to minimize the attacker's recognition accuracy, while the attacker seeks to maximize it. The expected cumulative reward of this interaction is denoted as R, reflecting the dynamic balance between defense effectiveness and communication performance.

3.3. MADRL-based modeling of WF attack and defense

Within the framework of WF attacks and defenses, attackers and defenders can be represented as two interacting agents. The defender agent continuously adapts traffic features in response to network dynamics and attack feedback, whereas the attacker agent optimizes its classification strategy based on observed traffic variations. This adversarial interaction inherently forms a decentralized partially observable Markov decision process (Dec-POMDP) [40], as illustrated in Fig. 2. Formally, the Dec-POMDP can be formulated as: $\mathcal{M} = (\mathcal{A}, S, \mathcal{A}_1, \mathcal{A}_2, P, O, R, \gamma)$, where A is the set of agents, including the defender and the attacker. S is the state space representing the global state of the network. $\mathcal{A}_i$ is the action space for agent i, with $i \in \{1, 2\}$(defender and attacker). $P(s' \mid s, a_1, a_2)$ is the state transition probability function, describing the probability of transitioning from s to s' under the joint action (a_1, a_2). O_i is the local observation space of agent i, describing what each agent can perceive from the environment. $\mathcal{R}_f(s, a_i)$ is the reward function, defining the immediate reward that an agent receives after taking action a_i in state s. $\gamma \in [0, 1]$ is the discount factor, which controls the weight of future rewards [19].

Each agent aims to maximize its own long-term cumulative reward by optimizing its policy. Specifically, the defender dynamically adjusts its traffic allocation strategy based on local observations to obfuscate fingerprint features, while the attacker attempts to infer the visited website from observed traffic patterns. Through continuous interaction, both agents refine their strategies to counter each other, forming a highly dynamic adversarial learning process. Through continuous interaction, both agents iteratively adapt their strategies to counter each other, forming a highly dynamic adversarial learning process. However, due to the strong coupling between agents' strategies, directly applying single-agent reinforcement learning algorithms often results in training instability and policy oscillations. To address this issue, we adopt multi-agent proximal policy optimization (MAPPO), a multi-agent extension of the proximal policy optimization (PPO) algorithm. MAPPO's core advantage lies in the centralized training with decentralized execution (CTDE) approach, where agents are trained using global state information but make decisions based only on local observations during execution.

The policy optimization objective in MAPPO is formulated as: $\mathcal{L}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(r_t(\theta)A_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right) A_t\right)\right]$, where $r_t(\theta)$ is the ratio between the new and old policy probabilities, A_t is the advantage function, and ϵ is the clipping parameter that limits large policy updates. To ensure stable learning, MAPPO also introduces a value function loss and an entropy regularization term. The overall loss function is: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{policy}} + c_1\mathcal{L}_{\text{value}} - c_2\mathcal{H}(\pi)$, where $\mathcal{L}_{\text{policy}}$ is the policy loss, $\mathcal{L}_{\text{value}}$ is the value function loss, $\mathcal{H}(\pi)$ is the entropy of the policy (a regularization term to promote exploration), and c_1 and c_2 are coefficients controlling the balance between these terms.

4. An innovative defense: RAMAD

In this section, we present the algorithm overview, RAMAD design and its training process.

4.1. Algorithm overview

We propose RAMAD, an adaptive traffic-splitting defense strategy based on MADRL to enhance the capability of anonymous communication systems against WF attacks. In this algorithm, we model the defender and attacker in WF attacks as two agents, the defender agent and the attacker agent. The defender learns how to split traffic across multiple paths, while the attacker learns to infer the communication target from the observed traffic pattern.

Fig. 3 illustrates the interaction process between the defender and the attacker agents under the MADRL framework. At each time step t, the environment state s_t transitions to s_{t+1} after both agents execute their actions. Specifically, the defender selects an action a_t^d based on its local observation o_t^d to perform traffic-splitting defense, while the attacker selects an action a_t^a based on o_t^a to conduct WF attacks. The joint actions (a_t^d, a_t^a) determine the evolution of the environment and affect the resulting observable traffic patterns.

After action execution, the environment returns rewards r_t^d and r_t^a to evaluate the effectiveness of the defense and attack strategies. The defender is encouraged to obfuscate traffic patterns, whereas the attacker aims to improve website inference performance. These rewards provide immediate feedback for policy optimization. The interaction generates transition tuples $(s_t, o_t^d, o_t^a, a_t^d, a_t^a, r_t^d, r_t^a, s_{t+1})$, which are used to train a centralized critic under the centralized training and decentralized execution (CTDE) paradigm. The critic estimates the joint value function to guide policy updates for both agents. Through iterative optimization with deep neural network-based policy approximation, the defender learns adaptive traffic-splitting strategies, while the attacker continuously improves its inference capability.

4.2. RAMAD design

The strategy primarily involves two core modeling components: traffic representation construction and reinforcement learning modeling.

(1) Input and traffic representation. To reduce feature redundancy and computational overhead, we employ a pure direction sequence as the basic representation of traffic, defined as $\{d_1, d_2, \ldots, d_n\}$ where $d_i \in \{-1, +1\}$. Here, $+1$ denotes outbound traffic from the client to the server, and -1 denotes inbound traffic from the server to the client. For agent inputs: the defender agent's input is $s_d = \{d_i\}_{i=1}^n$, representing the packet direction sequence within the current traffic

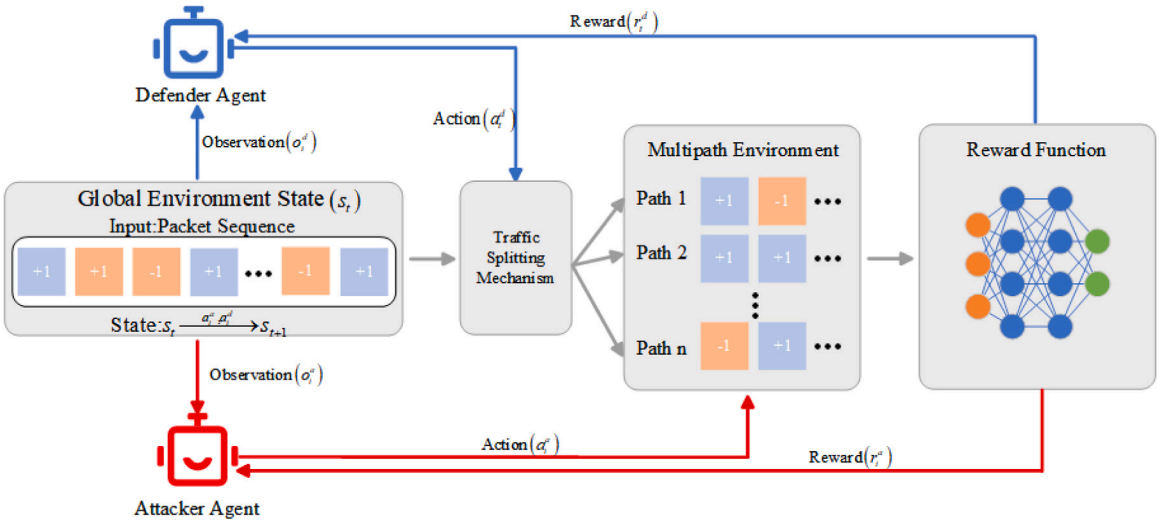


Fig. 3. The overview of RAMAD.

batch. The attacker agent's input is defined as $s_a = \{d_i\}_{i=1}^m$, representing the observed traffic sequence obtained by merging traffic from the subcircuits monitored by the attacker.

(2) Reinforcement learning modeling. The adversarial interaction between the defender and the attacker agents in this defense framework is modeled as a multi-agent reinforcement learning problem. The design is structured around three dimensions: state space, action space, and reward function: **Environment Definition and State Space.** The environmental state (i.e., network state at each time step) includes statistical features of the current traffic batch, path latency, and bandwidth information, represented as a vector:

$$s_t = \{Flow_1, Flow_2, \ldots, Flow_k, Congestion_1, Congestion_2, \ldots, Congestion_K, \ldots\}. \quad (1)$$

The defender observes partial network information, including basic features of the current traffic batch, historical attack feedback, path utilization, and latency measurements. Its observation space is defined as: $o_t^d = \{FlowStats_t, BWUtil_{t-1}, AttackFlag_{t-1}, Latency_t\}$. The attacker only observes a subset of monitored paths, including traffic features of the monitored paths and historical attack success rates. Its observation space is: $o_t^a = \{FlowStats_t, SuccessRate_{t-1}\}$.

Action space. The defender determines the probabilistic allocation of the current traffic batch across n available paths at each time step t. Its action space is a probability vector $a_t^d = p_t = \{p_1, p_2, \ldots, p_n\}$, where $p_i \in [0,1]$ is the probability of allocating the traffic batch to path i, satisfying $\sum_{i=1}^n p_i = 1$. The attacker's task is to infer the visited website by monitoring traffic on selected subcircuits and extracting distinguishing features; the attacker's action space therefore consists of choosing which subcircuits to monitor and which attack model to apply. Its action space is defined as $a_t^a = \{m_t, s_t\}$, consisting of two components:a mask vector $m_t = \{m_1, m_2, \ldots, m_n\}$(where $m_i \in \{0,1\}$ and $\sum_{i=1}^n m_i \le M$)indicating up to M subcircuits to monitor. An integer $s_t \in \{0,1,2,3\}$ selecting an attack model from a pretrained set S.

Reward function. Defender Reward integrates two components:

Security Reward r_{sec}^d reflects the success of attack obfuscation, defined as:

$$r_{\text{sec}}^d = \begin{cases} +1 & \text{if attack fails,} \\ -1 & \text{if attack succeeds,} \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

where the reward is only assigned at the end of a complete traffic interaction cycle; for intermediate (non-terminal) steps or incomplete sessions where the attack outcome cannot be determined, the reward is set to 0. The same definition applies to the attacker reward.

Uniformity reward r_{bal}^d promotes balanced traffic allocation across paths, given by

$$r_{\text{bal}}^d = -\sum_{i=1}^n p_i \log(p_i) \quad (3)$$

The final defender reward function is:

$$r_t^d = \alpha \cdot r_{\text{sec}}^d + \beta \cdot r_{\text{bal}}^d \quad (4)$$

α, β are weighting coefficients.

Attacker Reward consists of three components:

Attack Success Reward r_{sec}^a measures successful website identification, defined as:

$$r_{\text{sec}}^a = \begin{cases} +1 & \text{if attack succeeds,} \\ -1 & \text{if attack fails,} \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

Path Switching Penalty r_{switch}^a discourages frequent switching of monitored paths:

$$r_{\text{switch}}^a = -\sum_{i=1}^n |m_t(i) - m_{t-1}(i)| \quad (6)$$

Model Switching Penalty r_{model}^a discourages frequent switching of attack models:

$$r_{\text{model}}^a = \begin{cases} -1 & \text{if } s_t \neq s_{t-1}, \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

The final attacker reward function is:

$$r_t^a = \lambda_a \cdot r_{\text{sec}}^a + \lambda_m \cdot r_{\text{switch}}^a + \lambda_s \cdot r_{\text{model}}^a \quad (8)$$

$\lambda_a, \lambda_m, \lambda_s$ are weighting coefficients.

4.3. Training process

This section describes the joint training process of the defender and the attacker agents under the MADRL framework. To achieve dynamic strategy evolution and stable convergence, the MAPPO algorithm is adopted, with independent Actor–Critic network architectures constructed for each agent. The Actor network generates continuous action spaces (including the defender's traffic splitting ratios and the attacker's path/model selection) based on local observation inputs. The Critic network estimates global state values by taking the global

Algorithm 1 MAPPO-based MADRL training algorithm.

Input: MAPPO hyperparameters: number of episodes K, time steps T, discount factor γ, GAE parameter λ, PPO clipping factor ϵ, learning rate α, environment $\mathcal{E}$, reward functions.

Output: Trained Actor and Critic networks for all agents.

Initialize Actor parameters θ_i and Critic parameters ϕ_i for each agent i
for episode = 1 to K **do**
  Initialize environment and obtain initial global state s_0
  Empty trajectory buffer D
  for $t = 0$ to $T - 1$ **do**
    for each agent i **do**
      Observe local state o_t^i and sample action $a_t^i \sim \pi_{\theta_i}(a_t^i|o_t^i)$
    end for
    Execute joint action $\mathbf{a}_t = \{a_t^1, \ldots, a_t^N\}$ in environment
    Obtain next state s_{t+1} and rewards $\{r_t^i\}$
    Store $(s_t, \mathbf{a}_t, \{r_t^i\}, s_{t+1})$ into buffer D
  end for
  for each agent i **do**
    Compute advantages A_t^i using GAE with Critic $V_{\phi_i}(s_t)$
    Compute PPO loss:

$$L^{clip}(\theta_i) = \mathbb{E}_t\left[\min(r_t(\theta_i)A_t^i, \text{clip}(r_t(\theta_i), 1-\epsilon, 1+\epsilon)A_t^i)\right]$$

    Update Actor: $\theta_i \leftarrow \theta_i + \alpha\nabla_{\theta_i} L^{clip}(\theta_i)$
    Update Critic: $\phi_i \leftarrow \phi_i - \alpha\nabla_{\phi_i}(R_t^i - V_{\phi_i}(s_t))^2$
  end for
end for
return Trained Actor and Critic networks $\{\theta_i, \phi_i\}$

environmental state s_t and the joint action space a_t^i, producing a value estimate $V_{\phi_i}(s_t)$ to guide policy optimization and gradient updates. [1]

As outlined in Algorithm 1, the training process comprises the following key phases:

(1) Initialization phase. The Actor and Critic network parameters, denoted by θ and ϕ, are randomly initialized for both defender and attacker agents. In addition, the trajectory buffer D, the maximum number of training episodes K, and the trajectory sampling length T are established. Each episode corresponds to one complete learning iteration, during which agents interact with the environment for T time steps before network updates.

(2) Environment interaction and experience sampling. At the beginning of each training episode, each agent independently initializes hidden states of its Actor and Critic networks. Given the local observation o_t^i, the agent samples an action according to its current policy π_{θ_i}: $a_t^i \sim \pi_{\theta_i}(a_t^i|o_t^i)$. The environment then executes the joint action $\mathbf{at} = a_t^1, \ldots, a_t^N$ and returns the next global state $st + 1$, the local observations o_{t+1}^i, and the corresponding rewards r_t^i for each agent. The complete trajectory tuple $(s_t, \mathbf{at}, r_t^i, s_{t+1})$ is stored in the replay buffer D for subsequent training.

(3) Advantage estimation and cumulative return calculation. The advantage function $\hat{A}_t^i$ at each timestep is computed using generalized advantage estimation (GAE):

$$\hat{A}_t^i = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}^i \tag{9}$$

$$\delta_t^i = r_t^i + \gamma V_{\phi i}(o_{t+1}^i) - V_{\phi i}(o_t^i) \tag{10}$$

[1] MAPPO stands for Multi-Agent Proximal Policy Optimization. The algorithm uses the Proximal Policy Optimization method to optimize the policies of multiple agents in a reinforcement learning environment.

The future cumulative return $\hat{R}_t^i$ is calculated as:

$$\hat{R}_t^i = \sum_{l=0}^{T-t-1} \gamma^l r_{t+l}^i \tag{11}$$

(4) Actor policy network update. The policy parameters θ_i are optimized by minimizing the clipped PPO objective function:

$$L_{clip}(\theta_i) = \mathbb{E}_t\left[\min\left(r_t^i(\theta_i)\hat{A}_t^i, \text{clip}(r_t^i(\theta_i), 1-\epsilon, 1+\epsilon)\hat{A}_t^i\right)\right] \tag{12}$$

where the policy ratio between the updated and old policies is defined as:

$$r_t^i(\theta_i) = \frac{\pi\theta_i(a_t^i|o_t^i)}{\pi\theta_{old,i}(a_t^i|o_t^i)} \tag{13}$$

The Actor parameters are updated via gradient ascent as:

$$\theta_i \leftarrow \theta_i - \alpha_\pi \nabla_{\theta_i} L_{clip}(\theta_i) \tag{14}$$

(5) Critic value network update. The Critic network is trained to minimize the mean squared error between the predicted and target state values:

$$L_{VF}(\phi_i) = \frac{1}{2}\mathbb{E}_t\left[\left(V_{\phi_i}(o_t^i) - \hat{R}_t^i\right)^2\right] \tag{15}$$

and its parameters are updated as:

$$\phi_i \leftarrow \phi_i - \alpha_V \nabla_{\phi_i} L_{VF}(\phi_i) \tag{16}$$

5. Experiment and evaluation

In this section, we present the dataset and experimental setup used in our study, followed by a comprehensive evaluation of RAMAD's performance and accuracy based on real-world traffic traces.

5.1. Dataset and experimental setup

To systematically validate the effectiveness of RAMAD, we provide detailed specifications across six dimensions: experimental design, data sources, attack model configuration, evaluation metrics, experimental platform, and hyperparameter settings. This ensures the reproducibility of our experiments and the reliability of the results.

(1) Experimental design. We conduct a series of experiments to evaluate RAMAD from multiple perspectives. *(i) Training convergence analysis:* we monitor the reward curves and strategy stability during training to verify that the defense policy converges properly. *(ii) Reward function design and sensitivity analysis:* we examine the impact of different reward structures and weight combinations to study how reward design affects defense performance. *(iii) Path structure and monitoring budget:* we analyze how the number of available paths and the attacker's monitoring budget influence the attack success rate. *(iv) Strategy stability analysis:* we compare the learned defense strategy with both a uniform allocation baseline and a single-agent strategy under the same settings to assess whether RAMAD provides consistent performance gains. *(v) Comparative analysis with mainstream defense strategies:* we compare RAMAD with representative defenses, including Tamaraw [29], WTF-PAD [27], Minipatch [34], and FRONT [28], to evaluate its effectiveness in both privacy protection and performance. *(vi) Robustness against adaptive adversaries:* we evaluate whether RAMAD remains effective under adaptive attack settings, including generalization to unseen attack models, defense-aware retraining, and ensemble-based attacks. *(vii) Information leakage analysis:* we further quantify the privacy protection capability of RAMAD using the WeFDE framework. *(viii) Open-world setting analysis:* we assess the performance of RAMAD under the open-world scenario to verify its robustness in a more realistic deployment setting. *(ix) Computational overhead and deployment feasibility:* we measure the standalone inference latency and assess whether RAMAD can be deployed efficiently on common user

Table 2
Selected attack methods and their characteristics.

Attack type	Model name	Core features
Traditional machine learning	k-FP [21]	Non-deep baseline
	XGBoost	Tree-ensemble features
Deep learning	RF [23]	Aggregated temporal
	DF [6]	Temporal
	Tik-Tok [24]	Spatio-temporal fusion
	VarCNN	Variable-length CNN features

devices. *(x) Scalability with parallel environment count:* we study how different numbers of parallel training environments affect convergence time and final defense performance.

(2) Dataset. The experiments are conducted under both closed-world and open-world settings using the DF dataset [6]. The closed-world subset (D-CW) includes 95 monitored websites, with 1000 traces per site, collected from the Alexa top 100 via the Tor Browser. The open-world subset (D-OW) consists of 40,716 traces from the Alexa top 50,000 websites (excluding those in D-CW), with each site visited once and inaccessible pages filtered out. The path latency data used in our experiments is sourced from the real-world network measurement file `circuits_latencies_new.txt`, provided by Wladimir De la Cadena et al. [15]. Each record in this file represents the transmission latency of an individual subcircuit, providing realistic temporal dynamics for simulation.

(3) Attack model configuration. To simulate a diverse range of adversarial capabilities, we consider multiple representative attack models that cover both traditional machine learning and deep learning-based feature structures. Since different experiments in this paper serve different evaluation purposes, we use different attacker subsets accordingly. Specifically, the ablation study employs a representative set of classical and deep-learning-based attackers to quantify the contribution of each module in RAMAD, including RF, XGBoost, Tik-Tok, and Var-CNN; the comparative analysis follows the attacker settings commonly used by mainstream defense methods to ensure fair comparison, where we use k-FP, RF, DF, and Tik-Tok as benchmark attack models. We evaluate robustness against adaptive adversaries from three complementary perspectives. For generalization evaluation, we adopt a representative attacker set (RF, Tik-Tok, DF, k-FP) commonly used in prior work. We further validate robustness against additional unseen attacker families, including XGBoost and Var-CNN. For defense-aware retraining evaluation, we use a retraining-capable attacker set (RF, Tik-Tok, Var-CNN, XGBoost) to model on-the-fly adaptation to defended traces. For ensemble evaluation, we aggregate predictions from multiple attackers to simulate a stronger composite adversary. For the scalability study, we use the same representative attacker set as in the main experimental setting, including RF, XGBoost, Tik-Tok, and Var-CNN.

The selected attack models and their characteristics are summarized in Table 2.

(4) Evaluation metrics. Three evaluation metrics are defined to assess the proposed method from the perspectives of security, performance overhead, and strategy perturbation.

Attack success rate (ASR). This metric quantifies the effectiveness of the defense mechanism by measuring the proportion of user-accessed websites correctly identified by the attacker [17]. The formula is expressed as:

$$\mathrm{ASR} = \frac{N_{\mathrm{correct}}}{N_{\mathrm{total}}} \times 100\% \tag{17}$$

where N_{correct} denotes the number of traffic samples successfully recognized by the attacker, and N_{total} represents the total number of test samples. A lower ASR value indicates a stronger obfuscation effect, implying higher defensive robustness against WF attacks.

Time overhead (Delay). This metric evaluates the communication cost introduced by the defense mechanism, specifically defined as the percentage of artificially added latency (for traffic obfuscation) relative to the original transmission time [41]. The calculation formula is:

$$\mathrm{Delay} = \frac{T_{\mathrm{actual}} - T_{\mathrm{original}}}{T_{\mathrm{original}}} \times 100\% \tag{18}$$

Here, T_{actual} denotes the transmission delay under the defense strategy, while T_{original} refers to the baseline delay without any defense. This metric reflects the trade-off between security and communication efficiency, ensuring that latency overhead remains within a practically acceptable range.

Path allocation entropy (H). This metric captures the perturbation intensity and unpredictability of traffic allocation. Higher entropy values indicate a more balanced utilization of available paths, thereby increasing the difficulty of successful traffic classification by attackers [42]. The entropy is computed as:

$$H(p) = -\sum_{i=1}^{n} p_i \log_2 p_i \tag{19}$$

where p_i represents the probability of traffic being allocated to the ith path (satisfying $\sum_{i=1}^{n} p_i = 1$), and n is the total number of paths. An entropy value approaching $\log_2 n$ signifies an optimally balanced path allocation, leading to maximal defense effectiveness through enhanced obfuscation.

ROC curve under open-world setting. To evaluate defense performance in the open-world scenario, we adopt the receiver operating characteristic (ROC) curve to measure the separability between monitored and unmonitored traffic. Monitored samples are treated as the positive class, while unmonitored samples are treated as the negative class.

For a given decision threshold τ, the true positive rate (TPR) and false positive rate (FPR) are defined as

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{20}$$

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}} \tag{21}$$

where TP and FN denote the numbers of correctly and incorrectly classified monitored samples, respectively, and FP and TN denote the numbers of unmonitored samples incorrectly classified as monitored and correctly classified as unmonitored, respectively.

By varying the threshold τ, a series of $(\mathrm{FPR}, \mathrm{TPR})$ pairs are obtained to form the ROC curve. The area under the curve (AUC) serves as a threshold-independent metric to quantify the attacker's discriminative capability. A curve close to the diagonal baseline indicates that the attacker can barely distinguish monitored from unmonitored traffic, suggesting stronger defense performance.

(5) Experimental settings. To validate the effectiveness of MADRL framework against WF attacks, we conducted extensive simulation experiments. The core configurations and parameters of the experimental platform are summarized in Table 3.

Table 3
Experimental platform and core parameter settings.

Category	Configuration/Description
Hardware Configuration	Intel Xeon Gold 5218, 128 GB RAM
Software Platform	Ubuntu 18.04; Python 3.8.20; PyTorch 1.13.0
Reinforcement Learning Library	XuanCe-1.2.3
Network Architecture	Actor/Critic: MLP (2 layers, 128/64 neurons), activation: ReLU
Path Number	Fixed $n = 3$ (three independent paths per transmission)
Reward Function Weights	Defense: Security 2.0, Allocation entropy 0.1; Attack: Attack success 1.0, Path switching cost 0.1, Model switching cost 0.3

Table 4
Hyperparameter settings.

Category	Hyperparameter	Value
Environment	State dimension	12
	Action dimension	4
	Path number n	3
Network	Parallel environments	1
	Discount factor	0.99
	Mini-batch size	64
	Optimizer	Adam
	Learning rate	0.0007
	GAE parameter	0.95
	Clipping parameter	0.2
Training	Entropy loss coefficient	0.01
	Critic loss coefficient	0.5
	Steps per rollout	512
	Training iterations	1500

Distinct weighting coefficients were assigned in the reward functions to accurately represent the objectives of the defender and the attacker during the adversarial training process. The coefficients were empirically determined through extensive experiments to achieve a balance between defense performance and training stability. For the defender agent, three weights were configured as follows: a security reward weight of 2.0 to emphasize defense effectiveness; an allocation entropy reward weight of 0.1 to moderately discourage high-latency paths. For the attacker agent, the reward function consisted of three weights: an attack success reward weight of 1.0 to enhance classification accuracy; a path switching penalty weight of 0.1 to discourage frequent path changes; and a model switching penalty weight of 0.3 to maintain model stability.

In each experiment, data transmission was performed through three independent network paths to simplify the model's input dimension and improve policy stability and reproducibility. Consequently, the number of paths was fixed at $n = 3$ throughout all experiments. The detailed hyperparameter settings are summarized in Table 4.

5.2. Training convergence analysis

In this section, we evaluate the training feasibility and strategy stability of RAMAD. The convergence behaviors of both the defender and the attacker agents are analyzed by tracking the evolution of their average cumulative rewards during training.

From Fig. 4, it can be observed that the defender's reward remains at a high level in the early training stage, mainly because the attacker's strategy has not yet been effectively learned, resulting in a low overall attack success rate. However, as training progresses, the attacker gradually improves its classification ability through continuous interactions, which challenges the defense strategy and leads to a gradual decrease in the defender's reward. After approximately 60 training iterations, the reward stabilizes, indicating that the defense strategy has largely completed its environmental exploration and entered the convergence phase, thereby demonstrating a certain level of stability and robustness.

As depicted in Fig. 5, the attacker's reward is initially low since its strategy starts in a random exploratory state. With continued training, the attacker progressively learns the defender's behavioral patterns, and its reward increases, reflecting the attacker's learning ability and adaptability. However, in the later stages of training, the reward curve does not show a continuous upward trend but instead stabilizes with small oscillations, indicating that the defender's agent still effectively interferes with the attack through continuous optimization. Eventually, both the defender and attacker reach a certain equilibrium in the adversarial game. During the early stages of training, both agents engage in active exploration, resulting in noticeable reward fluctuations. However, after the introduction of the GAE advantage estimation and PPO clipping update mechanism, the amplitude of these fluctuations significantly decreases, demonstrating improved training stability and a well-balanced adversarial interaction.

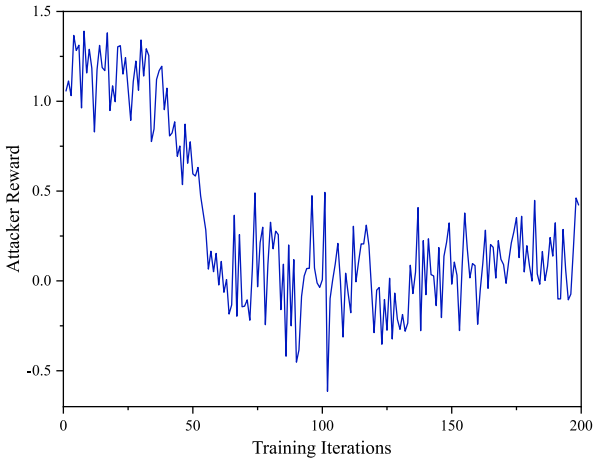


Fig. 4. Defender's average cumulative reward curve.

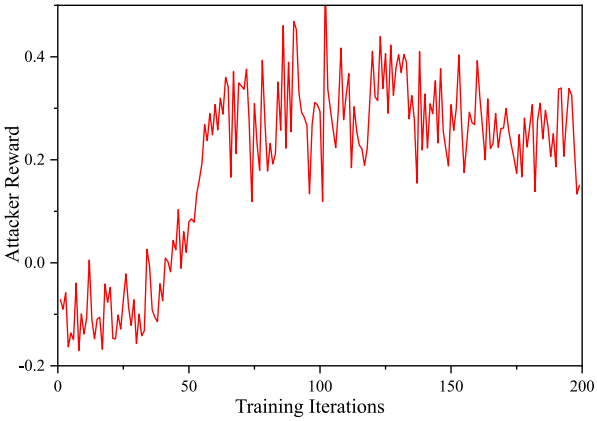


Fig. 5. Attacker's average cumulative reward curve.

5.3. Reward function design and sensitivity analysis

To stabilize adversarial training, the defender reward is formulated as a weighted combination of two components: a security term that reflects attack success, and an entropy-based term that encourages balanced traffic allocation across paths. This design prioritizes privacy protection while promoting routing diversity. The attacker reward remains fixed across all experiments, allowing the impact of the defender-side reward design to be isolated.

To evaluate the robustness of the proposed formulation, we conduct a sensitivity analysis over five defender reward configurations: *security only*, *baseline*, *strong entropy*, *weak security*, and *strong security*. Specifically, the *security only* setting removes the entropy term; the *baseline* uses ($\alpha = 2.0, \beta = 0.1$); the *strong entropy* setting increases the entropy weight to $\beta = 0.2$; the *weak security* setting reduces the security weight to $\alpha = 1.0$; and the *strong security* setting increases the security weight to $\alpha = 3.0$. All experiments are trained under the same protocol, with only the defender reward coefficients varied.

Fig. 6 presents the attack success rate and average entropy under different configurations. The baseline achieves the lowest attack success rate, indicating the best privacy protection among all settings. In contrast, the *security only* variant exhibits inferior performance, suggesting that the entropy term plays a critical role in obscuring traffic patterns by encouraging diversified routing. Furthermore, simply increasing either the security weight or the entropy weight does not consistently improve performance, indicating that excessive emphasis on a single objective may disrupt the balance between obfuscation and stability.

Table 5
Attack success rates under different numbers of available paths.

n	RF	XGBoost	Tik-Tok	var_cnn	Avg.
2	0.5937	0.1921	0.4267	0.6434	0.4640
3	0.3530	0.1811	0.2035	0.3072	0.2612
4	0.3307	0.1385	0.1808	0.2630	0.2283
5	0.3556	0.1294	0.2034	0.3671	0.2639

Table 6
Performance overhead under different numbers of available paths.

n	Avg. Latency	Std. Latency	Latency Range
2	0.09660	0.00384	[0.0841, 0.1183]
3	0.09653	0.00379	[0.0842, 0.1186]
4	0.09652	0.00378	[0.0840, 0.1186]
5	0.09641	0.00376	[0.0840, 0.1184]

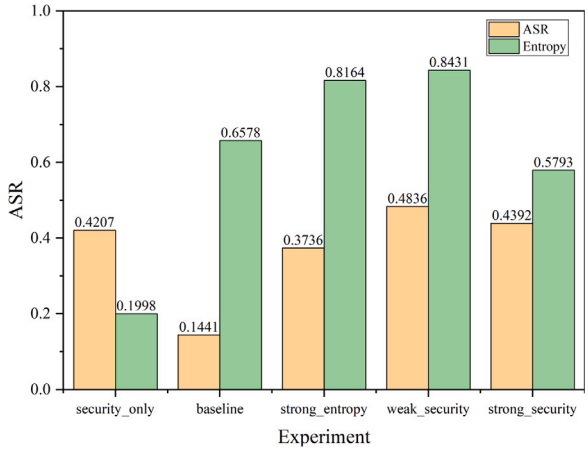


Fig. 6. Reward sensitivity analysis.

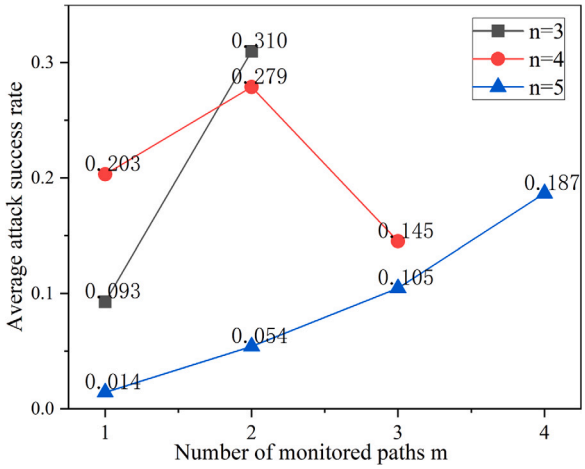


Fig. 7. Attack success rate versus the number of monitored paths m under different numbers of available paths n.

Overall, these results demonstrate that both the security and entropy terms are essential to the defense objective, and that an appropriate balance between them is crucial. The baseline configuration provides the best trade-off between privacy protection and traffic diversity within the tested parameter range.

5.4. Path structure and monitoring budget

To systematically evaluate the impact of routing diversity, we vary the number of available paths as $n = 2, 3, 4, 5$. These values are chosen to cover a spectrum from minimal to moderate path diversity. Specifically, $n = 2$ represents the simplest multi-path setting and serves as a baseline, while $n = 3$ and $n = 4$ reflect practical configurations with moderate diversity. The case $n = 5$ is further included to examine whether increasing the path space continues to provide security benefits or leads to diminishing returns due to increased strategy complexity. For each value of n, the attacker's monitored-path budget m is further varied to examine how the available path structure and the monitoring capability jointly affect attack success and communication overhead.

As shown in Table 5, increasing the number of available paths from $n = 2$ to $n = 4$ consistently reduces the attack success rate across all evaluated models, with the average success rate decreasing from 46.40% at $n = 2$ to 26.12% at $n = 3$, and further to 22.83% at $n = 4$. This indicates that routing diversity can effectively disperse traffic and weaken the burst-level and path-specific patterns exploited by website fingerprinting attacks. However, the benefit becomes non-monotonic when $n = 5$, where the average attack success rate slightly rises compared with $n = 4$. This suggests diminishing returns in routing diversity: while a larger routing space provides more flexibility for traffic splitting, it also enlarges the defender's action space and increases the difficulty of learning a stable policy, so additional paths do not necessarily translate into proportional privacy gains.

Table 6 further shows that communication latency remains highly stable across different values of n. The average latency varies only marginally, indicating that increasing routing diversity does not introduce noticeable overhead in our implementation. This is an important practical property, as it suggests that RAMAD can improve security without sacrificing communication efficiency. Taken together, the security and overhead results indicate that $n = 3$ offers a favorable trade-off, while $n = 4$ achieves slightly better security at the cost of a larger routing space. Based on these observations, we select $n = 3$ as the default configuration in the main experiments, since it already provides substantial privacy improvement over the two-path case while maintaining low overhead and stable training behavior.

To further examine the effect of the attacker's monitoring budget, we analyze attack success under different m for each path configuration. As shown in Fig. 7, the relationship is not strictly monotonic. For $n = 5$, the attack success rate generally increases with m, since a larger budget allows the attacker to observe more paths and extract discriminative traffic features. However, the marginal benefit diminishes for very large m, as additional observations tend to be fragmented and redundant. In contrast, for $n = 4$, the success rate first rises and then falls with increasing m. Initially, more monitored paths improve observability, but beyond a moderate m, redundant or inconsistent observations reduce the effectiveness of any single discriminative pattern. This non-monotonicity reflects the interaction between the attacker's monitoring capacity and RAMAD's adaptive traffic-splitting policy, showing that inference performance depends on both factors rather than monitoring budget alone.

Overall, these results demonstrate that attack effectiveness is jointly determined by the available path structure and the attacker's learned monitoring policy. By treating the monitored-path selection as part of the attacker's action space, RAMAD allows the monitoring behavior to evolve adaptively during training. This design better captures the coupled dynamics between defense decisions and attack observations, leading to a more realistic adversarial evaluation.

5.5. Strategy stability analysis

To further evaluate the policy performance, Fig. 8 presents a comparative analysis of the multi-agent strategy, the single-agent strategy, and the uniform allocation strategy in terms of attack success rate, communication latency, and traffic allocation entropy. In this evaluation,

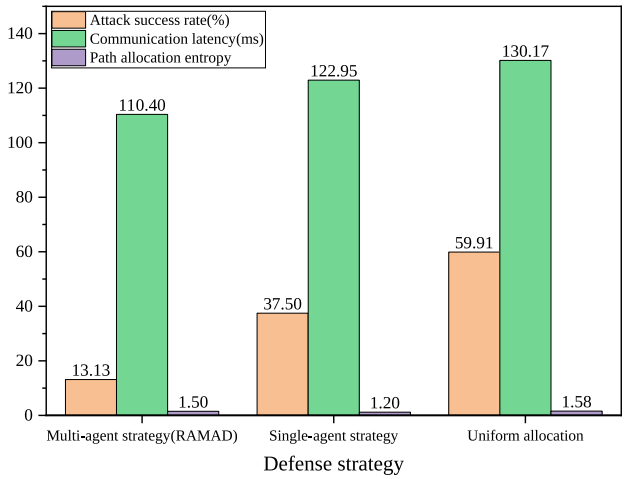


Fig. 8. Performance comparison results.

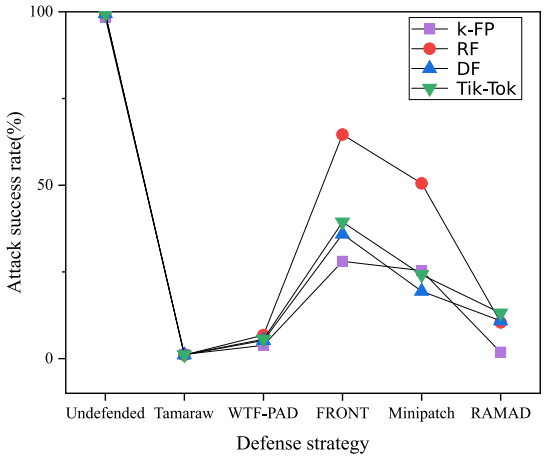


Fig. 9. Comparison of attack success rates under different defense strategies.

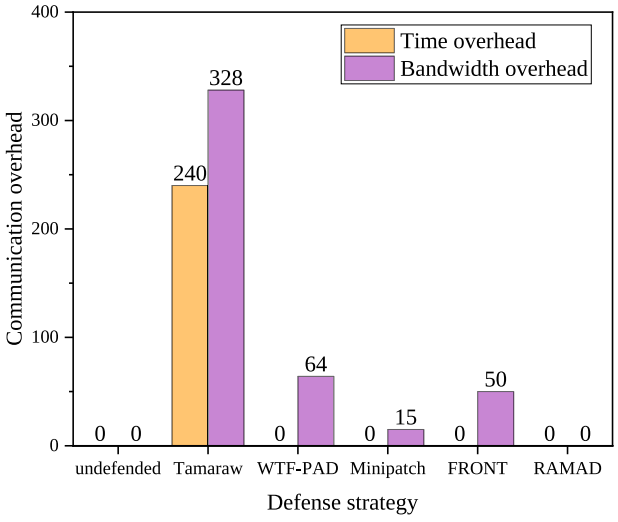


Fig. 10. Communication overhead under different defense strategies.

multiple attack models are employed to assess each defense strategy, and for each case, the best-performing attack result (i.e., the one with the highest classification accuracy) is adopted as the indicator of the corresponding defense security level.

In terms of attack success rate, RAMAD achieves a value of 13.13%, representing a reduction of approximately 46 percentage points (76.8%) compared to the uniform allocation strategy and 21.4 percentage points relative to the single-agent strategy. This substantial decrease confirms the strong defense capability and robust adaptability introduced by the multi-agent framework. Regarding communication latency, the MAPPO strategy maintains the lowest average delay of 110.40 ms, indicating that it effectively enhances security without compromising transmission efficiency. As for traffic allocation entropy, the MAPPO strategy reaches 1.50, which is close to that of the uniform allocation value (1.58) and significantly higher than that of the PPO strategy. This indicates that MAPPO exhibits greater randomness and unpredictability in path selection, thereby improving the traffic obfuscation effect and resistance to fingerprinting attacks.

Overall, RAMAD not only achieves stable convergence during training, but also demonstrates superior performance across all three evaluation metrics, highlighting its comprehensive advantage in dynamic adversarial environments.

5.6. Comparative analysis with mainstream defense strategies

As shown in Fig. 9 and Table 7, to comprehensively assess the adversarial robustness and communication performance of the proposed strategy, four representative defense mechanisms are selected as baselines for horizontal comparison: Tamaraw (regularization-based): enforces a fixed transmission rate to enhance traffic uniformity; WTF-PAD and FRONT (random perturbation-based): introduce dummy packets to obscure traffic patterns; and Minipatch (adversarial perturbation-based): leverages deep learning models to generate adversarial perturbations that interfere with fingerprinting classifiers. The experiments are conducted under four representative attack models — k-FP, RF, DF, and Tik-Tok — to evaluate the robustness and adaptability of the proposed strategy across diverse adversarial environments.

Fig. 10 compares the performance of different defense strategies in terms of defense effectiveness and communication overhead. As shown, Tamaraw achieves the strongest defensive capability but incurs the highest bandwidth and latency cost, which make it less practical for real-world deployment. WTF-PAD, FRONT, and Minipatch achieve better communication efficiency compared to Tamaraw; however, their overall defense strength remains limited, particularly under deep-learning-based attacks such as Tik-Tok and RF, where their performance exhibits noticeable fluctuations. In contrast, the proposed RAMAD strategy achieves an excellent defense success rate of only 13.13%, without introducing additional bandwidth or time overhead, thereby striking a desirable balance between robustness and efficiency.

Table 7
Attack success rates (%) of different fingerprinting models under various defense strategies.

Defense strategy	k-FP	RF	DF	Tik-Tok
Undefended	98.45	99.35	99.44	99.36
Tamaraw	1.20	1.04	1.07	1.01
WTF-PAD	6.78	5.23	5.60	6.86
FRONT	28.05	64.60	35.89	39.38
Minipatch	19.42	50.54	25.37	24.19
RAMAD (Proposed)	**10.41**	**13.13**	**10.88**	**10.83**

Note: Values represent the attack success rates (%) of different website fingerprinting models. Lower values indicate stronger defense effectiveness.

5.7. Robustness against adaptive adversaries

Generalization evaluation. To evaluate the robustness and generalization ability of the proposed defense strategy against unseen attack models, a generalization attack experiment was conducted. In this

Table 8
Attack success rate of RAMAD under unknown attack models.

Test attack model	Attack success Rate (%)
XGBoost	14.61
Var-CNN	15.92

Note: The table reports the attack success rates (%) of unseen attack models when evaluated against the RAMAD.

Table 9
Defense-aware retraining evaluation results under defended traffic. Lower online attack success rate indicates stronger immediate defense, while higher after-adaptation accuracy indicates stronger attacker recovery.

Attacker	After-adaptation accuracy	Online attack success rate
Tik-Tok	0.9522	0.2259
RF	0.9085	0.2910
XGBoost	0.4774	0.2267
Var-CNN	0.6728	0.3832

experiment, the attacker adopts previously unseen attack models during testing to assess the cross-model adaptability of the defense policy. Specifically, during the training phase, the attacker agents employ four known attack models — k-FP, RF, DF, and Tik-Tok — to participate in the reinforcement learning process. During the testing phase, two new attack models that were never introduced during training are incorporated to evaluate the generalization performance: XGBoost — a lightweight ensemble-based boosting model widely used for high-performance classification tasks; Var-CNN — a deep learning-based attack model integrating attention mechanisms with convolutional architectures, specifically designed for WF attacks.

As shown in Table 8, the attack success rate under the XGBoost attack is 14.61%, and 15.92% under the Var-CNN attack. Compared with the strongest attack during training, Tik-Tok (13.13%), the increase is only about 1.5–2.8 percentage points, indicating that the proposed strategy does not overfit the training attack models. This result demonstrates that the defense perturbations exhibit cross-model generalization ability, effectively resisting novel attack classifiers with unknown structures. Furthermore, since XGBoost and Var-CNN respectively represent lightweight and heavyweight attack models, the proposed RAMAD strategy maintains a consistently low attack success rate across both cases. This finding highlights its structure-agnostic robustness and strong generalization capability in heterogeneous adversarial environments.

Defense-aware retraining evaluation. We evaluate RAMAD under an adaptive threat model where the attacker is allowed to retrain on defended traffic after observing the defense-induced distribution shift. We consider a retraining-capable attack model set including RF, Tik-Tok, Var-CNN, and XGBoost. Each attack model is first tested in its pretrained form on the defended split, and then fine-tuned on a subset of defended traces before being re-evaluated on the held-out samples. We then use these adaptive attackers to evaluate RAMAD's robustness against defense-aware adversaries.

As shown in Table 9, the online attack success rate measures the adaptive attacker's immediate performance against RAMAD, while the after-adaptation accuracy measures the recovered performance after fine-tuning on defended traces. The results indicate that RAMAD substantially suppresses online attack success rate, and although fine-tuning can partially recover attacker performance, the recovery is still constrained. This suggests that RAMAD remains effective against defense-aware attackers that adapt to the defended distribution.

Table 10
Ensemble evaluation results under defended traffic.

Metric	Accuracy
Average single attacker	0.0985
Best single attacker	0.1024
Majority-voting ensemble attacker	0.1961

Ensemble evaluation. We further evaluate RAMAD under an ensemble attack setting to simulate a stronger adversary. Instead of relying on a single classifier, the attacker aggregates predictions from multiple base models and determines the final label by majority voting. This setting simulates a more practical and more powerful attacker that can leverage complementary decision boundaries across different model families to improve robustness.

Table 10 compares the average single-attacker accuracy, the best single-attacker accuracy, and the majority-voting ensemble accuracy. These three metrics jointly characterize the strength of the coordinated adversary: the average value reflects the overall capability of the base attackers, the best single-attacker accuracy indicates the upper bound of a standalone attacker, and the ensemble accuracy captures the final attack success after model fusion.

The results show that combining multiple attackers can strengthen the adversary by exploiting model diversity. However, RAMAD still limits the effectiveness of the coordinated attack, as reflected by the ensemble accuracy. This indicates that the defense remains robust even when the attacker is allowed to fuse multiple base models into a stronger joint decision mechanism.

5.8. Information leakage analysis

To further characterize the privacy-preserving capability of RAMAD, we conduct an information-theoretic analysis of feature-level distinguishability using the WeFDE framework [43]. WeFDE quantifies the amount of information that a traffic feature f reveals about a webpage label w via mutual information. Following this methodology, a set of commonly used WF features is extracted, redundant features are removed, and the remaining features are ranked according to their leakage contribution. We focus on both the maximum mutual information of individual features and the distribution of the most informative features.

Following the WeFDE protocol, we report four metrics: the average mutual information (Avg MI), the total mutual information (Total MI), the top-1 mutual information (Top-1 MI), and the median mutual information of the top 100 most informative features (Top-100 Median MI). Avg MI and Total MI characterize the overall leakage level, while Top-1 MI captures the most vulnerable feature and Top-100 Median MI measures the typical leakage strength among highly informative features.

Table 11 presents the comparison results. RAMAD consistently reduces information leakage compared with the undefended baseline, and outperforms Tamaraw, Minipatch, and FRONT across all metrics. In particular, the reductions in Avg MI and Total MI indicate that RAMAD effectively suppresses overall feature leakage, while the lower Top-1 MI demonstrates its ability to mitigate the strongest leakage channel. Although WTF-PAD attains slightly lower leakage in this setting, RAMAD remains competitive and provides a more balanced performance across different metrics.

Overall, these results confirm that RAMAD significantly reduces traffic fingerprint leakage and enhances privacy protection from an information-theoretic perspective.

Table 11
Information leakage comparison under the WeFDE framework. Lower values indicate better privacy protection.

Method	Avg MI	Total MI	Top-1 MI	Top-100 Median MI
Undefended	0.0310	2.6991	0.0522	0.0339
Tamaraw	0.0302	2.6286	0.0500	0.0333
WTF-PAD	0.0247	2.1508	0.0441	0.0266
Minipatch	0.0287	2.4940	0.0512	0.0306
FRONT	0.0301	2.6166	0.0460	0.0336
RAMAD	**0.0275**	**2.3906**	**0.0461**	**0.0311**

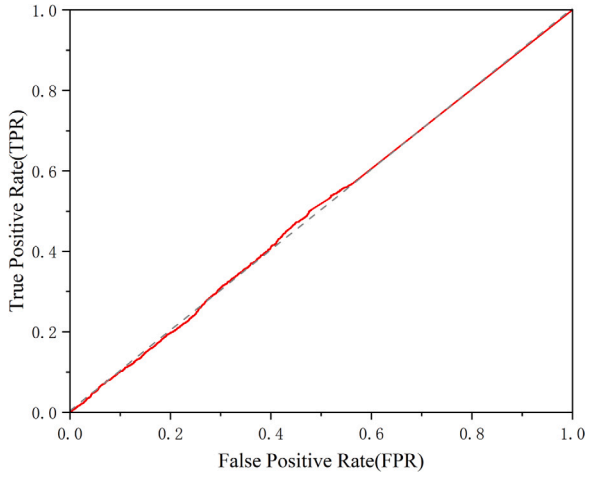


Fig. 11. ROC curve in the open-world scenario.

5.9. Open-world setting analysis

To further assess the robustness of RAMAD in a more realistic deployment scenario, we conduct an open-world binary classification evaluation, where the attacker distinguishes between *monitored* and *unmonitored* traffic. In our open-world setup, the monitored set contains 95,000 traces from 95 monitored websites with 1000 traces per site, while the unmonitored set contains 40,716 traces collected from unseen websites outside the training distribution. The monitored classes correspond to website categories observed during training, while the unmonitored classes represent traffic samples outside the training distribution. Compared with the closed-world setting, this scenario better reflects practical conditions in which unseen traffic patterns are present.

The ROC curve in Fig. 11 is used to measure the separability between monitored and unmonitored traffic. The curve lies very close to the diagonal reference line, indicating that the attacker's discriminative capability is close to random guessing. This observation is further confirmed by an ROC-AUC of 0.5046, which is only marginally above 0.5. Such a result suggests that the proposed defense effectively reduces the statistical distinguishability between monitored and unmonitored traffic distributions. By obscuring discriminative traffic features, RAMAD prevents the attacker from establishing a reliable decision boundary, thereby significantly degrading its inference capability in the open-world setting.

5.10. Computational overhead and deployment feasibility

To assess the computational overhead of RAMAD and its deployment feasibility on client-side devices, we conduct a lightweight profiling study on a CPU-only platform described in the Experimental settings section, equipped with an Intel Xeon processor.

Table 12 summarizes several metrics of RAMAD. The policy forward time reflects the inference cost of the policy network, the per-step decision time measures the time needed to generate an action, the environment step time captures the simulation-side transition cost, the per-step execution time represents the total time required for one decision cycle, and the memory footprint measures the overall memory usage during execution.

Table 12
Runtime profiling results of RAMAD on CPU.

Metric	Mean	Std
Policy forward time (ms)	0.61	0.11
Per-step decision time (ms)	1.24	0.04
Environment step time (ms)	12.42	5.21
Per-step execution time (ms)	13.66	5.21
Memory footprint (MB)	3826.41	0.00

Table 13
Scalability of RAMAD with different numbers of parallel environments.

Parallel Envs	Convergence time (h)	Attack success rate (%)
1	4.92	26.12
2	3.54	26.82
4	2.45	26.22
8	1.83	25.42
16	1.46	24.71

The policy forward pass requires 0.61 ms on average, and the overall decision time is 1.24 ms, indicating that the learned policy can be executed at sub-millisecond to millisecond latency. The environment step accounts for the majority of the per-step execution time, with an average of 12.42 ms, resulting in a total per-step execution time of 13.66 ms. Since the environment step corresponds to simulation overhead rather than deployment-time inference, the actual online cost is dominated by the policy forward pass. The reported memory usage reflects the complete experimental setup, including the simulation environment and auxiliary components. The policy network itself is lightweight and can be deployed independently with a substantially smaller memory footprint.

We further evaluate the standalone policy inference latency on a laptop equipped with an Intel Core i7-8565U and a smartphone equipped with a Kirin 9010. The average inference latency is 1.5 ms and 3.6 ms, respectively. Under the same input representation as used in our CPU profiling, the policy inference consistently remains in the low-millisecond range across both devices.

Overall, these results suggest that RAMAD imposes low inference overhead on resource-constrained client-side devices. This indicates that RAMAD is feasible for practical deployment on common user devices, although more comprehensive device-specific profiling remains future work.

5.11. Scalability with parallel environment count

RAMAD is evaluated in a single-environment setting in the main experiments for simplicity and clarity. Nevertheless, it is naturally extensible to parallel environment execution, which enables more efficient experience collection and scalable training.

To assess the scalability of RAMAD, we further study the effect of the number of parallel environments, including 1, 2, 4, 8, and 16. We measure both convergence time and attack success rate under each setting. As shown in Table 13, increasing the number of parallel environments can substantially improve training throughput and accelerate the optimization process. Meanwhile, the attack success rate remains largely stable across all settings and exhibits a slight downward trend as the number of parallel environments increases, suggesting that RAMAD preserves its defense effectiveness under different levels of parallelism. These results show that RAMAD can leverage parallel RL training to speed up convergence while maintaining stable defense performance.

6. Conclusion and future work

This paper proposes an adaptive traffic splitting defense strategy, named RAMAD, based on MADRL to counter deep WF attacks in anonymous communication systems. Both the defender and attacker are modeled as heterogeneous agents engaged in an adversarial Markov decision process. Using a CTDE framework with the MAPPO algorithm, the two agents evolve their strategies, enabling the defender to perceive attack changes and flexibly adjust traffic allocation to enhance defense performance while maintaining latency and load balance.

Comprehensive experiments on real-world Tor traffic and typical attack models show that RAMAD significantly outperforms existing defenses in attack accuracy reduction, communication overhead, and disturbance balance. Against deep learning-based attacks, it lowers the attack success rate by over 80%, demonstrating strong robustness, adaptability, and model-agnostic capability.

Future work will focus on four directions: **(1) Real-world deployment.** Future research can focus on embedding the defender agent into real communication proxy systems to explore its stability and practicality under conditions such as bandwidth fluctuations, delay jitter, and real traffic scenarios. **(2) Single-path extension.** RAMAD is currently designed for multi-path scenarios, where the defender learns adaptive traffic-splitting policies against website fingerprinting attacks. A natural extension is to generalize the MADRL-based framework to single-path environments, where the action space shifts from path allocation to alternative perturbation mechanisms, such as timing modulation, packet padding, traffic shaping, or dummy traffic injection. This would extend the applicability of RAMAD to more constrained network settings. **(3) Enhanced attack diversity.** Subsequent work can expand the capabilities of the attacker agents, introducing more complex attack paradigms for adversarial training, further improving the defense strategy's generalization ability and adaptability to real-world scenarios. **(4) Temporal-aware defense.** Although RAMAD currently adopts a compact direction-only observation space, an important extension is to incorporate temporal information into the defender's observations, such as packet inter-arrival times, burst-level statistics, and packet timing vectors.

In summary, this work establishes a multi-agent game-theoretic framework for adaptive WF defense and validates its theoretical and experimental effectiveness. Future work will further improve the framework by incorporating richer temporal representations, extending it to more diverse deployment scenarios, and validating its practicality in real anonymous communication systems.

CRediT authorship contribution statement

Yufei Zhu: Writing – original draft, Validation, Conceptualization. **Le Tian:** Writing – review & editing, Validation, Supervision. **Yuxiang Hu:** Project administration, Formal analysis. **Xinglong Pei:** Resources, Methodology, Data curation. **Qi Zhan:** Visualization, Validation. **Qilong Huang:** Visualization, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the Science and Technology Innovation leading Talents Subsidy Project of Central Plains under Grant 244200510038.

Data availability

Data will be made available on request.

References

[1] R. Dingledine, N. Mathewson, P. Syverson, Tor: The Second-Generation onion router, in: 13th USENIX Security Symposium, USENIX Security 04, USENIX Association, San Diego, CA, 2004, URL https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router.

[2] M. Alsabah, I. Goldberg, Performance and security improvements for tor: A survey, ACM Comput. Surv. 49 (2) (2016) http://dx.doi.org/10.1145/2946802.

[3] C.H. Komlo, N. Mathewson, I. Goldberg, Walking onions: Scaling anonymity networks while protecting users, in: 29th USENIX Security Symposium, USENIX Security 20, USENIX Association, 2020, pp. 1003–1020, URL https://www.usenix.org/conference/usenixsecurity20/presentation/komlo.

[4] G. Cherubin, R. Jansen, C. Troncoso, Online website fingerprinting: Evaluating website fingerprinting attacks on tor in the real world, in: 31st USENIX Security Symposium, USENIX Security 22, USENIX Association, Boston, MA, 2022, pp. 753–770, URL https://www.usenix.org/conference/usenixsecurity22/presentation/cherubin.

[5] X. Zhao, X. Deng, Q. Li, Y. Liu, Z. Liu, K. Sun, K. Xu, Towards fine-grained webpage fingerprinting at scale, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 423–436, http://dx.doi.org/10.1145/3658644.3690211.

[6] P. Sirinam, M. Imani, M. Juarez, M. Wright, Deep fingerprinting: Undermining website fingerprinting defenses with deep learning, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 1928–1943, http://dx.doi.org/10.1145/3243734.3243768.

[7] X. Xiao, X. Zhou, Z. Yang, L. Yu, B. Zhang, Q. Liu, X. Luo, A comprehensive analysis of website fingerprinting defenses on tor, Comput. Secur. 136 (2024) 103577, http://dx.doi.org/10.1016/j.cose.2023.103577, URL https://www.sciencedirect.com/science/article/pii/S016740482300487X.

[8] T. Wang, The one-page setting: A higher standard for evaluating website fingerprinting defenses, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 2794–2806, http://dx.doi.org/10.1145/3460120.3484790.

[9] A. Niakanlahiji, S. Orlowski, A. Vahid, J.H. Jafarian, Toward practical defense against traffic analysis attacks on encrypted DNS traffic, Comput. Secur. 124 (2023) 103001, http://dx.doi.org/10.1016/j.cose.2022.103001, URL https://www.sciencedirect.com/science/article/pii/S0167404822003935.

[10] D. Lu, S. Bhat, A. Kwon, S. Devadas, DynaFlow: An efficient website fingerprinting defense based on dynamically-adjusting flows, in: Proceedings of the 2018 Workshop on Privacy in the Electronic Society, WPES '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 109–113, http://dx.doi.org/10.1145/3267323.3268960.

[11] T. Wang, X. Cai, R. Nithyanand, R. Johnson, I. Goldberg, Effective attacks and provable defenses for website fingerprinting, in: Proceedings of the 23rd USENIX Conference on Security Symposium, SEC '14, USENIX Association, USA, 2014, pp. 143–157.

[12] M. Nasr, A. Bahramali, A. Houmansadr, Defeating DNN-based traffic analysis systems in real-time with blind adversarial perturbations, in: 30th USENIX Security Symposium, USENIX Security 21, USENIX Association, 2021, pp. 2705–2722, URL https://www.usenix.org/conference/usenixsecurity21/presentation/nasr.

[13] J. Gong, W. Zhang, C. Zhang, T. Wang, Surakav: Generating realistic traces for a strong website fingerprinting defense, in: 2022 IEEE Symposium on Security and Privacy, SP, 2022, pp. 1558–1573, http://dx.doi.org/10.1109/SP46214.2022.9833722.

[14] K.P. Dyer, S.E. Coull, T. Ristenpart, T. Shrimpton, Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail, in: Proceedings of the 2012 IEEE Symposium on Security and Privacy, SP '12, IEEE Computer Society, USA, 2012, pp. 332–346, http://dx.doi.org/10.1109/SP.2012.28.

[15] W. De la Cadena, A. Mitseva, J. Hiller, J. Pennekamp, S. Reuter, J. Filter, T. Engel, K. Wehrle, A. Panchenko, TrafficSliver: Fighting website fingerprinting attacks with traffic splitting, in: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1971–1985, http://dx.doi.org/10.1145/3372297.3423351.

[16] W. Du, S. Ding, A survey on multi-agent deep reinforcement learning: from the perspective of challenges and applications, Artif. Intell. Rev. 54 (2021) 3215–3238, http://dx.doi.org/10.1007/s10462-020-09938-y, URL https://link.springer.com/article/10.1007/s10462-020-09938-y.

[17] N. Mathews, J.K. Holland, S.E. Oh, M.S. Rahman, N. Hopper, M. Wright, Sok: A critical evaluation of efficient website fingerprinting defenses, in: 2023 IEEE Symposium on Security and Privacy, SP, 2023, pp. 969–986, http://dx.doi.org/10.1109/SP46215.2023.10179289.

[18] E. Erdin, C. Zachor, M.H. Gunes, How to find hidden users: A survey of attacks on anonymity networks, IEEE Commun. Surv. Tutor. 17 (4) (2015) 2296–2316, http://dx.doi.org/10.1109/COMST.2015.2453434.

[19] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, Y. Wu, The surprising effectiveness of PPO in cooperative multi-agent games, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), in: Advances in Neural Information Processing Systems, vol. 35, Curran Associates, Inc., 2022, pp. 24611–24624, URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9c1535a02f0ce079433344e14d910597-Paper-Datasets_and_Benchmarks.pdf.

[20] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, K. Wehrle, Website fingerprinting at internet scale, in: NDSS, vol. 1, 2016, p. 23477.

[21] J. Hayes, G. Danezis, K-fingerprinting: A robust scalable website fingerprinting technique, in: 25th USENIX Security Symposium, USENIX Security 16, USENIX Association, Austin, TX, 2016, pp. 1187–1203, URL https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/hayes.

[22] K. Al-Naami, A. El-Ghamry, M.S. Islam, L. Khan, B. Thuraisingham, K.W. Hamlen, M. Alrahmawy, M.Z. Rashad, Bimorphing: A bi-directional bursting defense against website fingerprinting attacks, IEEE Trans. Dependable Secur. Comput. 18 (2) (2021) 505–517, http://dx.doi.org/10.1109/TDSC.2019.2907240.

[23] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, K. Xu, Subverting website fingerprinting defenses with robust traffic representation, in: 32nd USENIX Security Symposium, USENIX Security 23, USENIX Association, Anaheim, CA, 2023, pp. 607–624, URL https://www.usenix.org/conference/usenixsecurity23/presentation/shen-meng.

[24] M.S. Rahman, P. Sirinam, N. Mathews, K.G. Gangadhara, M. Wright, Tik-tok: The utility of packet timing in website fingerprinting attacks, Proc. Priv. Enhancing Technol. (PoPETs) 2020 (3) (2020) 5–24, http://dx.doi.org/10.2478/popets-2020-0043, URL https://petsymposium.org/popets/2020/popets-2020-0043.php.

[25] J. Gong, W. Cai, S. Liang, Z. Guan, T. Wang, E.-C. Chang, WFCAT: Augmenting website fingerprinting with channel-wise attention on timing features, IEEE Trans. Dependable Secur. Comput. 23 (1) (2026) 149–163, http://dx.doi.org/10.1109/TDSC.2025.3605197.

[26] X. Hong, X. Ma, S. Li, H. Qiu, B. Liu, A website fingerprint defense technology with low delay and controllable bandwidth, Comput. Commun. 193 (2022) 332–345, http://dx.doi.org/10.1016/j.comcom.2022.06.028, URL https://www.sciencedirect.com/science/article/pii/S0140366422002304.

[27] M. Juárez, M. Imani, M. Perry, C. Dıaz, M. Wright, WTF-PAD: toward an efficient website fingerprinting defense for tor, 2015, CoRR, abs/1512.00524.

[28] J. Gong, T. Wang, Zero-delay lightweight defenses against website fingerprinting, in: 29th USENIX Security Symposium, USENIX Security 20, USENIX Association, 2020, pp. 717–734, URL https://www.usenix.org/conference/usenixsecurity20/presentation/gong.

[29] X. Cai, R. Nithyanand, T. Wang, R. Johnson, I. Goldberg, A systematic approach to developing and evaluating website fingerprinting defenses, in: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, CCS '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 227–238, http://dx.doi.org/10.1145/2660267.2660362.

[30] M.S. Rahman, M. Imani, N. Mathews, M. Wright, Mockingbird: Defending against deep-learning-based website fingerprinting attacks with adversarial traces, IEEE Trans. Inf. Forensics Secur. 16 (2021) 1594–1609, http://dx.doi.org/10.1109/TIFS.2020.3039691.

[31] C. Hou, G. Gou, J. Shi, P. Fu, G. Xiong, WF-GAN: Fighting back against website fingerprinting attack using adversarial learning, in: 2020 IEEE Symposium on Computers and Communications, ISCC, 2020, pp. 1–7, http://dx.doi.org/10.1109/ISCC50000.2020.9219593.

[32] J. Huang, W. Liu, G. Liu, B. Gao, F. Nie, WF-A2D: Enhancing privacy with asymmetric adversarial defense against website fingerprinting, IEEE Trans. Inf. Forensics Secur. 20 (2025) 4739–4754, http://dx.doi.org/10.1109/TIFS.2025.3565994.

[33] S. Henri, G. García-Avilés, P. Serrano, A. Banchs, P. Thiran, Protecting against website fingerprinting with multihoming, in: Proceedings on Privacy Enhancing Technologies (PoPETs) / PETS 2020, (2) 2020, pp. 89–110, http://dx.doi.org/10.2478/popets-2020-0019, URL https://petsymposium.org/popets/2020/popets-2020-0019.php.

[34] D. Li, Y. Zhu, M. Chen, J. Wang, Minipatch: Undermining DNN-based website fingerprinting with adversarial patches, IEEE Trans. Inf. Forensics Secur. 17 (2022) 2437–2451, http://dx.doi.org/10.1109/TIFS.2022.3186743.

[35] J. Guo, H. Gao, Z. Liu, F. Huang, J. Zhang, X. Li, J. Ma, ICRA: An intelligent clustering routing approach for UAV ad hoc networks, IEEE Trans. Intell. Transp. Syst. 24 (2) (2023) 2447–2460, http://dx.doi.org/10.1109/TITS.2022.3145857.

[36] J. Guo, X. Li, Z. Liu, J. Ma, C. Yang, J. Zhang, D. Wu, TROVE: A context-awareness trust model for VANETs using reinforcement learning, IEEE Internet Things J. 7 (7) (2020) 6647–6662, http://dx.doi.org/10.1109/JIOT.2020.2975084.

[37] L. Basyoni, A. Erbad, A. Mohamed, M. Guizani, QDRL: QoS-aware deep reinforcement learning approach for Tor's circuit scheduling, IEEE Trans. Netw. Sci. Eng. 9 (5) (2022) 3396–3410, http://dx.doi.org/10.1109/TNSE.2022.3179569.

[38] M. Jiang, B. Cui, J. Fu, T. Wang, Z. Wang, KimeraPAD: A novel low-overhead real-time defense against website fingerprinting attacks based on deep reinforcement learning, IEEE Trans. Netw. Serv. Manag. 21 (3) (2024) 2944–2961, http://dx.doi.org/10.1109/TNSM.2024.3360082.

[39] I. Karunanayake, N. Ahmed, R. Malaney, R. Islam, S.K. Jha, De-anonymisation attacks on tor: A survey, IEEE Commun. Surv. Tutor. 23 (4) (2021) 2324–2350, http://dx.doi.org/10.1109/COMST.2021.3093615.

[40] I. Chadès, L.V. Pascal, S. Nicol, C.S. Fletcher, J. Ferrer-Mestres, A primer on partially observable Markov decision processes (POMDPs), Methods Ecol. Evol. 12 (11) (2021) 2058–2072, http://dx.doi.org/10.1111/2041-210X.13692.

[41] X. Xiao, X. Zhou, Z. Yang, L. Yu, B. Zhang, Q. Liu, X. Luo, A comprehensive analysis of website fingerprinting defenses on tor, Comput. Secur. 136 (C) (2024) http://dx.doi.org/10.1016/j.cose.2023.103577.

[42] Z. Xu, C. Pu, J. Yang, Link prediction based on path entropy, Phys. A 456 (2016) 294–301, http://dx.doi.org/10.1016/j.physa.2016.03.091, URL https://www.sciencedirect.com/science/article/pii/S0378437116300899.

[43] S. Li, H. Guo, N. Hopper, Measuring information leakage in website fingerprinting attacks and defenses, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 1977–1992, http://dx.doi.org/10.1145/3243734.3243832.